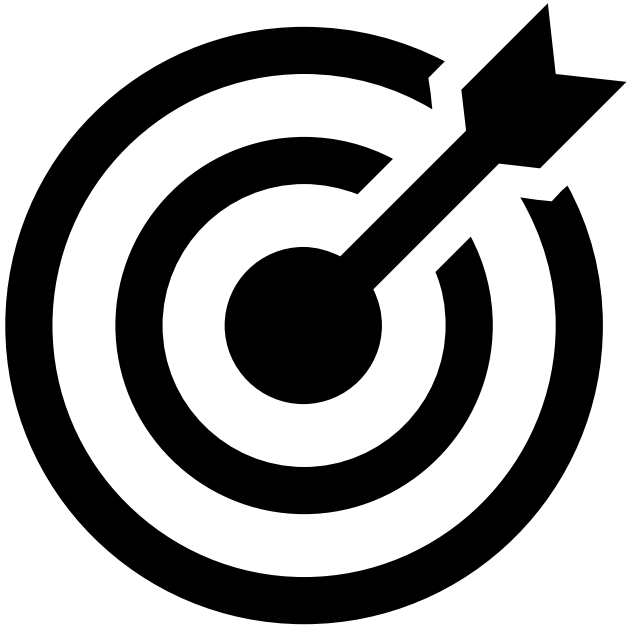
RECHNERARCHITEKTUR
Übungsstunde 1

Daniel Große
Simon Reitinger, Katharina Ruep
Institute for Complex Systems (ICS)

daniel.grosse@jku.at
simon.reitinger@jku.at, katharina.ruep@jku.at

AGENDA



- Organisatorisches
  - Prüfungsleistung
  - Übungsabgaben und Projekt
  - Gruppenarbeit
  - Literatur
- Tools & Helper
  - EPICS
  - VENUS
  - RISC-V Reference Card
- Assembler Beispiel
- Übungszettel 1

- Ziel: Übersicht über Übungsbetrieb, Einblick in die Tools und erstes gemeinsames Beispiel

PRÜFUNGSLEISTUNG

Vorlesung

- Klausurtermin:
  Mittwoch 24. Juni 2026
  um 13:45
  (bitte freihalten!)

Übung

- Übungszettel (i.d.R. wöchentlich)
- In der Regel:
  24 Punkte pro Übungsblatt
- Projekt (2. Teil des Semesters)
  48 Punkte

Bewertung

- 50% der Punkte zum erfolgreichen Abschluss
- Anzahl(Abgaben) > 2 impliziert Bewertung

Noten

- ≥ 87.5%: 1 (sehr gut)
- ≥ 75%: 2 (gut)
- ≥ 62.5%: 3 (befriedigend)
- ≥ 50%: 4 (genügend)

ÜBUNGSABGABEN & PROJEKT

- Bewertung durch Tutoren
- Ausgabe der Übungsblätter begleitend zu den Vorlesungskapiteln
- Abgabe der Übungsblätter
  - **Donnerstags 10:15** (**vor** der 1. Übungsgruppe; min. 7 Tage Bearbeitungszeit)
  - Verspätete Abgabe (max. 12h): Max. 50% der Punkte erreichbar
- Projekt hat spezielles Abgabedatum
- Ausnahmen (z.B. wegen Feiertag) werden angegeben;
  exaktes Abgabedatum findet sich **immer** auf den Übungsblättern
- Abgaben in *EPICS*
- Keine Punkte bei Plagiaten, Betrugsversuchen

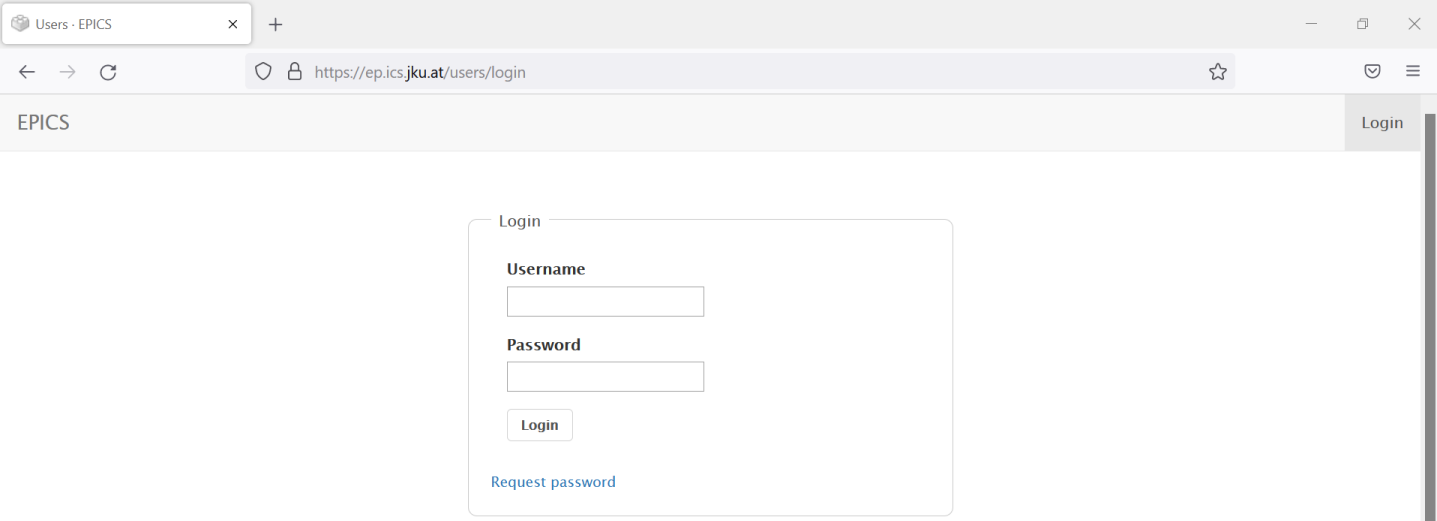
GRUPPENARBEIT

- Gruppengröße: 3 Personen
- Alle Gruppenmitglieder sollen Lösungen nachvollziehen können (spätestens für Klausur relevant)
- Gruppenbildung
  - in Thread im Moodle Forum
  - bis spätestens **Sonntag, 15.03.2026**
- Änderung einer Gruppe
  - bis jeweils **Samstag, 23:59 Uhr** nach Ausgabe eines Übungszettels möglich
  - in Moodle (bitte informiert vorab alle Gruppenmitglieder)
- Je Gruppe nur eine Abgabe!
- Jedes Gruppenmitglied MUSS aktiv an Abgaben mitarbeiten (ggf. Gruppenänderung)

LITERATUR

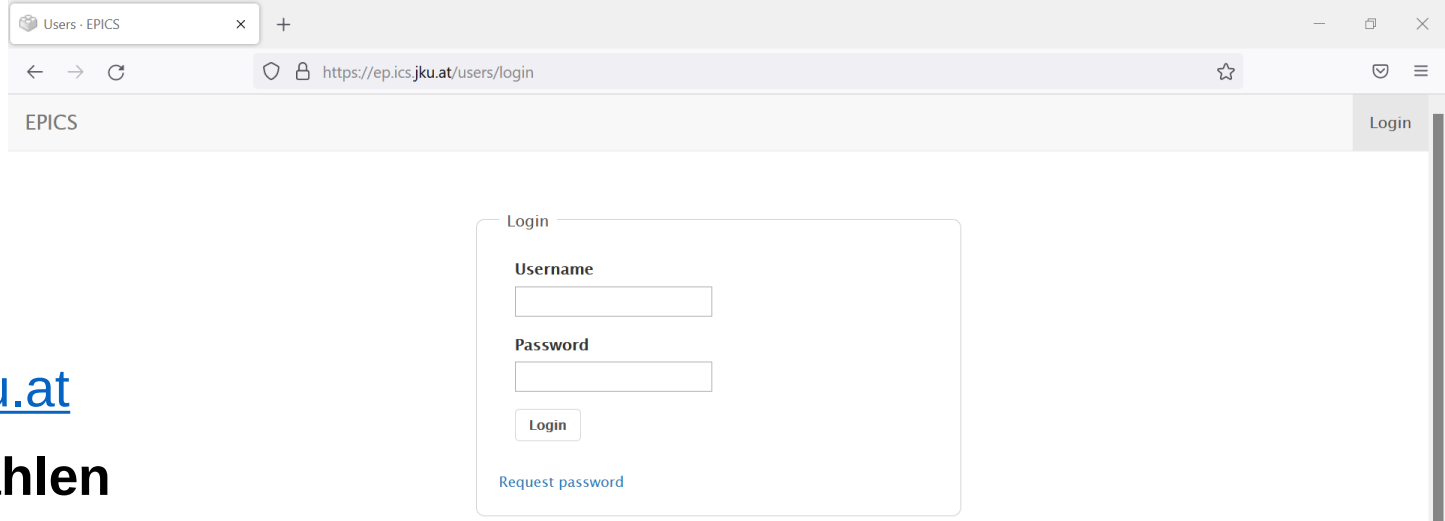
- Sarah L. Harris, David Harris, “Digital design and computer architecture – RISC-V Edition”, Morgan Kaufmann, 2021.
- David A. Patterson, John L. Hennessy, “Computer Organization and Design RISC-V Edition: The Hardware Software Interface”, 2. Edition, 2021
- Becker/Drechsler/Molitor, „Technische Informatik – Eine Einführung“, Pearson Studium, 2005 (Überarbeitung: Becker/Molitor, Oldenbourg Verlag, 2008)
- D. Hoffmann, „Grundlagen der Technischen Informatik“, Hanser Verlag, 2007
- David A. Patterson, John L. Hennessy, Arndt Bode, Wolfgang Karl, Theo Ungerer, „Rechnerorganisation und -entwurf”, Morgan Kaufmann Publishers, 2005.
- Sivarama P. Dandamudi, “Fundamentals of Computer Organization and Design”, Springer, 2004.

EP.ICS



- https://ep.ics.jku.at
- Klick auf **Request password**
  - Email Adresse eingeben (k.....@students.jku.at)
- Man erhält ein Email mit einem Link um das Passwort zu setzen
- Einloggen

ÜBUNGSABGABE



- Bei EPICS einloggen: https://ep.ics.jku.at
- **Rechnerarchitektur SS26 UE auswählen**
- Die Dateien mit der Ausarbeitung in den Workspace hochladen
- Hochgeladene Dateien und die zugehörige Aufgabe auswählen
- Auf **submit** klicken (nur Hochladen ist noch ***keine*** Übungsabgabe)
- Unter **My Submissions** sieht man die abgegebenen Ausarbeitungen
- Bis zur Deadline kann die Abgabe überarbeitet werden
- Die Bewertung findet man unter **My Grades**

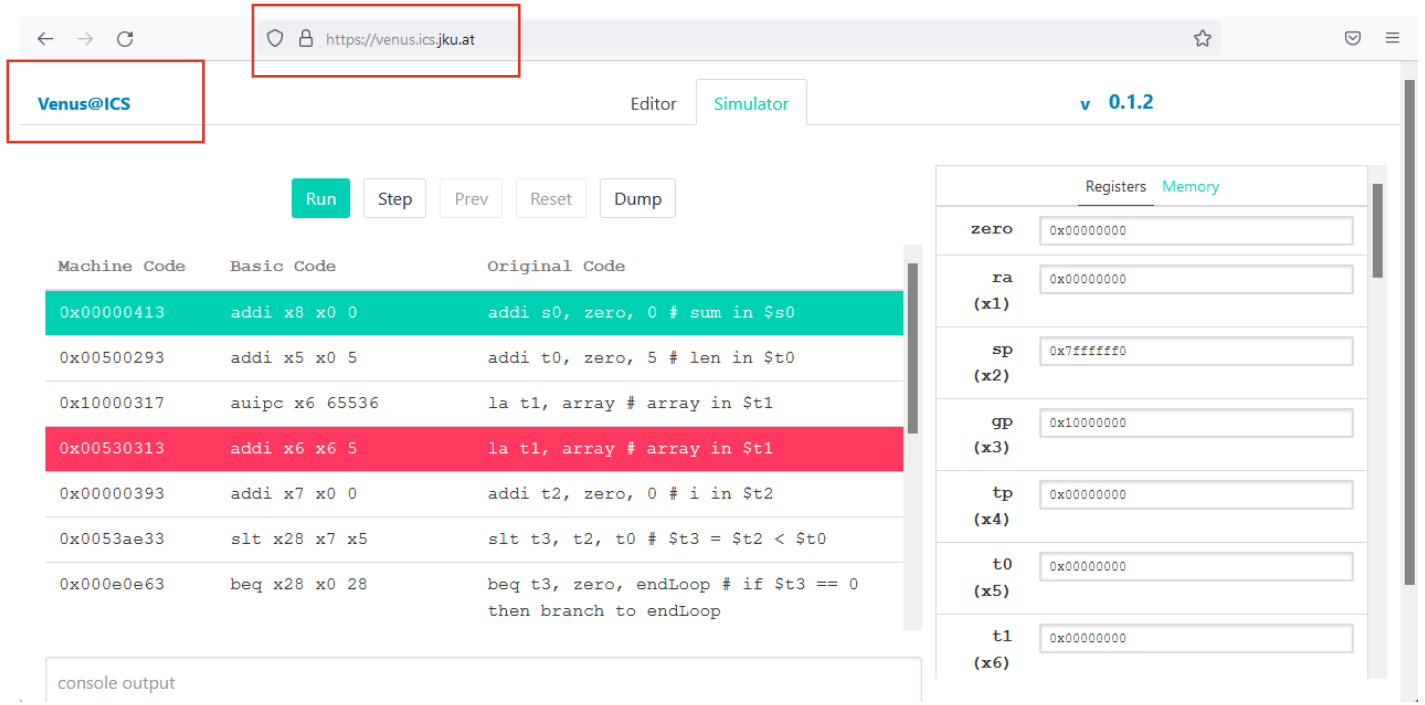
- Nur eine Abgabe pro Gruppe!

ÜBUNGSZETTEL 0

- Bonusübung für EPICS
- 2 Bonuspunkte gruppenunabhängig
- Wie immer zu finden in Moodle

VENUS

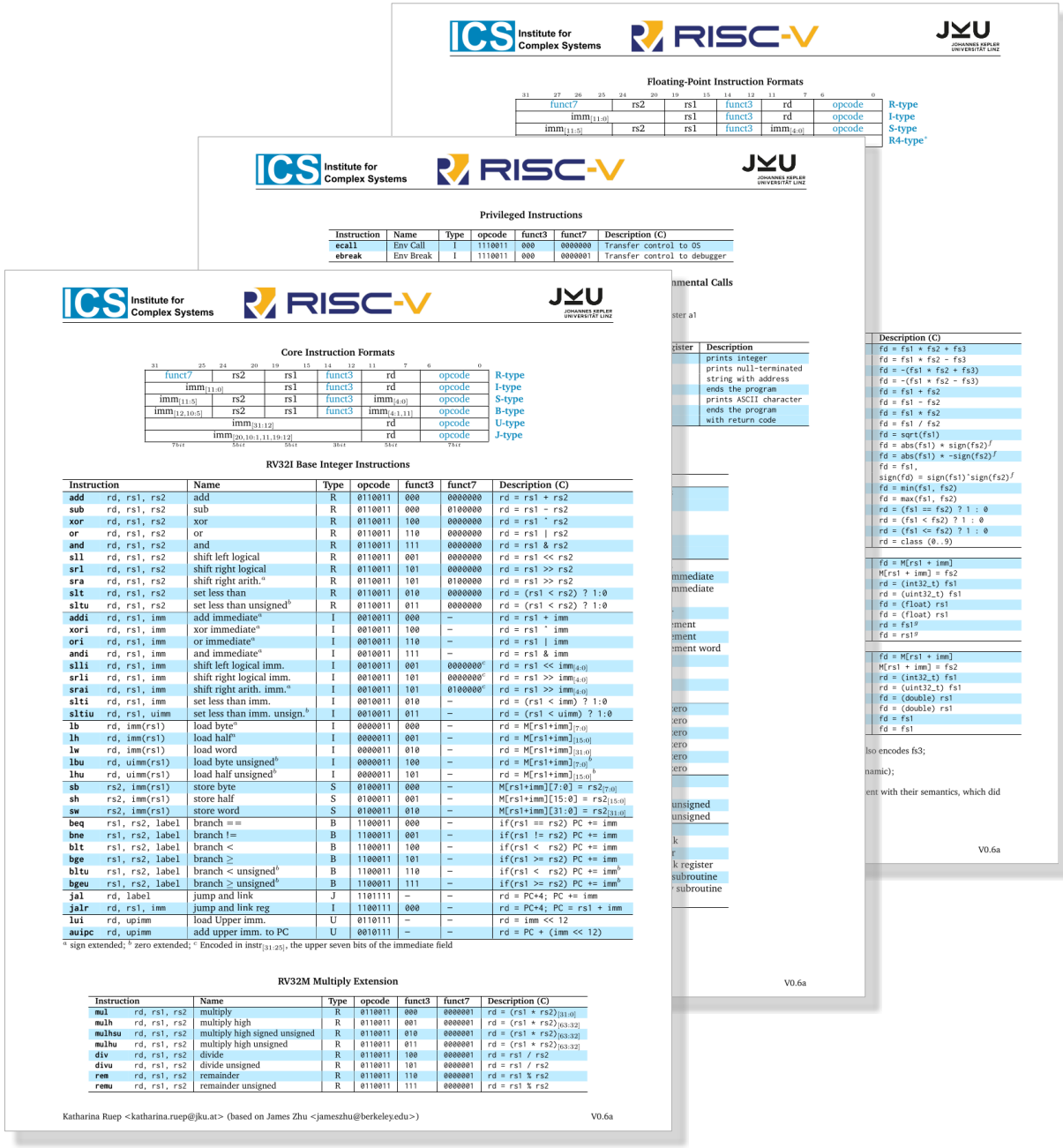
- Venus ist ein Simulator für RISC-V
  - https://venus.ics.jku.at/



- Zeigt Zustand von Registern und Speicher an
- Schrittweises Ausführen von Programmen
  - Ermöglicht Debugging

RISC-V REFERENCE CARD

- “Cheat Sheet“ mit allen wichtigen Referenzen für diese Übung
  - Formate
  - Instruktionen (Base, Multiply, Floating-Point)
  - Register
- In Moodle zu finden



ICS Institute for Complex Systems — RISC-V — JKU Johannes Kepler Universität Linz

Floating-Point Instruction Formats

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
$imm_{[11:0]}$			rs1	funct3	rd	opcode	I-type
$imm_{[11:5]}$		rs2	rs1	funct3	$imm_{[4:0]}$	opcode	S-type
$imm_{[12,10:5]}$		rs2	rs1	funct3	$imm_{[4:1,11]}$	opcode	R4-type[+]
$imm_{[31:12]}$					rd	opcode	U-type
$imm_{[20,10:1,11,19:12]}$					rd	opcode	J-type
7bit	5bit	5bit	3bit	5bit	7bit		

Privileged Instructions

Instruction	Name	Type	opcode	funct3	funct7	Description (C)
ecall	Env Call	I	1110011	000	0000000	Transfer control to OS
ebreak	Env Break	I	1110011	000	0000001	Transfer control to debugger

Core Instruction Formats

31 25	24 20	19 15	14 12	11 7	6 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
$imm_{[11:0]}$		rs1	funct3	rd	opcode	I-type
$imm_{[11:5]}$	rs2	rs1	funct3	$imm_{[4:0]}$	opcode	S-type
$imm_{[12,10:5]}$	rs2	rs1	funct3	$imm_{[4:1,11]}$	opcode	B-type
$imm_{[31:12]}$				rd	opcode	U-type
$imm_{[20,10:1,11,19:12]}$				rd	opcode	J-type
7bit	5bit	5bit	3bit	5bit	7bit	

RV32I Base Integer Instructions

Instruction		Name	Type	opcode	funct3	funct7	Description (C)
add	rd, rs1, rs2	add	R	0110011	000	0000000	rd = rs1 + rs2
sub	rd, rs1, rs2	sub	R	0110011	000	0100000	rd = rs1 - rs2
xor	rd, rs1, rs2	xor	R	0110011	100	0000000	rd = rs1 ^ rs2
or	rd, rs1, rs2	or	R	0110011	110	0000000	rd = rs1 | rs2
and	rd, rs1, rs2	and	R	0110011	111	0000000	rd = rs1 & rs2
sll	rd, rs1, rs2	shift left logical	R	0110011	001	0000000	rd = rs1 << rs2
srl	rd, rs1, rs2	shift right logical	R	0110011	101	0000000	rd = rs1 >> rs2
sra	rd, rs1, rs2	shift right arith.[a]	R	0110011	101	0100000	rd = rs1 >> rs2
slt	rd, rs1, rs2	set less than	R	0110011	010	0000000	rd = (rs1 < rs2) ? 1:0
sltu	rd, rs1, rs2	set less than unsigned[b]	R	0110011	011	0000000	rd = (rs1 < rs2) ? 1:0
addi	rd, rs1, imm	add immediate[a]	I	0010011	000	–	rd = rs1 + imm
xori	rd, rs1, imm	xor immediate[a]	I	0010011	100	–	rd = rs1 ^ imm
ori	rd, rs1, imm	or immediate[a]	I	0010011	110	–	rd = rs1 | imm
andi	rd, rs1, imm	and immediate[a]	I	0010011	111	–	rd = rs1 & imm
slli	rd, rs1, imm	shift left logical imm.	I	0010011	001	0000000[c]	rd = rs1 << $imm_{[4:0]}$
srli	rd, rs1, imm	shift right logical imm.	I	0010011	101	0000000[c]	rd = rs1 >> $imm_{[4:0]}$
srai	rd, rs1, imm	shift right arith. imm.[a]	I	0010011	101	0100000[c]	rd = rs1 >> $imm_{[4:0]}$
slti	rd, rs1, imm	set less than imm.	I	0010011	010	–	rd = (rs1 < imm) ? 1:0
sltiu	rd, rs1, uimm	set less than imm. unsign.[b]	I	0010011	011	–	rd = (rs1 < uimm) ? 1:0
lb	rd, imm(rs1)	load byte[a]	I	0000011	000	–	rd = M[rs1+imm]$_{[7:0]}$
lh	rd, imm(rs1)	load half[a]	I	0000011	001	–	rd = M[rs1+imm]$_{[15:0]}$
lw	rd, imm(rs1)	load word	I	0000011	010	–	rd = M[rs1+imm]$_{[31:0]}$
lbu	rd, uimm(rs1)	load byte unsigned[b]	I	0000011	100	–	rd = M[rs1+imm]$_{[7:0]}$[b]
lhu	rd, uimm(rs1)	load half unsigned[b]	I	0000011	101	–	rd = M[rs1+imm]$_{[15:0]}$[b]
sb	rs2, imm(rs1)	store byte	S	0100011	000	–	M[rs1+imm][7:0] = $rs2_{[7:0]}$
sh	rs2, imm(rs1)	store half	S	0100011	001	–	M[rs1+imm][15:0] = $rs2_{[15:0]}$
sw	rs2, imm(rs1)	store word	S	0100011	010	–	M[rs1+imm][31:0] = $rs2_{[31:0]}$
beq	rs1, rs2, label	branch ==	B	1100011	000	–	if(rs1 == rs2) PC += imm
bne	rs1, rs2, label	branch !=	B	1100011	001	–	if(rs1 != rs2) PC += imm
blt	rs1, rs2, label	branch <	B	1100011	100	–	if(rs1 < rs2) PC += imm
bge	rs1, rs2, label	branch ≥	B	1100011	101	–	if(rs1 >= rs2) PC += imm
bltu	rs1, rs2, label	branch < unsigned[b]	B	1100011	110	–	if(rs1 < rs2) PC += imm[b]
bgeu	rs1, rs2, label	branch ≥ unsigned[b]	B	1100011	111	–	if(rs1 >= rs2) PC += imm[b]
jal	rd, label	jump and link	J	1101111	–	–	rd = PC+4; PC += imm
jalr	rd, rs1, imm	jump and link reg	I	1100111	000	–	rd = PC+4; PC = rs1 + imm
lui	rd, upimm	load Upper imm.	U	0110111	–	–	rd = imm << 12
auipc	rd, upimm	add upper imm. to PC	U	0010111	–	–	rd = PC + (imm << 12)

[a] sign extended; [b] zero extended; [c] Encoded in $inst_{[31:25]}$, the upper seven bits of the immediate field

RV32M Multiply Extension

Instruction		Name	Type	opcode	funct3	funct7	Description (C)
mul	rd, rs1, rs2	multiply	R	0110011	000	0000001	rd = $(rs1 * rs2)_{[31:0]}$
mulh	rd, rs1, rs2	multiply high	R	0110011	001	0000001	rd = $(rs1 * rs2)_{[63:32]}$
mulhsu	rd, rs1, rs2	multiply high signed unsigned	R	0110011	010	0000001	rd = $(rs1 * rs2)_{[63:32]}$
mulhu	rd, rs1, rs2	multiply high unsigned	R	0110011	011	0000001	rd = $(rs1 * rs2)_{[63:32]}$
div	rd, rs1, rs2	divide	R	0110011	100	0000001	rd = rs1 / rs2
divu	rd, rs1, rs2	divide unsigned	R	0110011	101	0000001	rd = rs1 / rs2
rem	rd, rs1, rs2	remainder	R	0110011	110	0000001	rd = rs1 % rs2
remu	rd, rs1, rs2	remainder unsigned	R	0110011	111	0000001	rd = rs1 % rs2

Katharina Ruep <katharina.ruep@jku.at> (based on James Zhu <jameszhu@berkeley.edu>) V0.6a

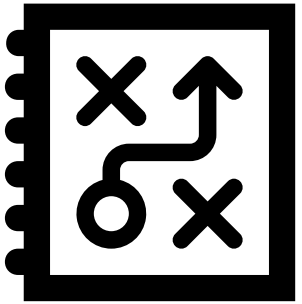
ASSEMBLER BEISPIEL

- Übersetzte den folgenden C Code in RISC-V Assembler mit Hilfe von VENUS
  - Bis auf la sind keine Pseudobefehle erlaubt

```
char string[] = "Hello World!\n“;
int array[] = {34,561,7,1,124};

int main() {
    int sum = 0;
    int len = 5;
    int i = 0;
    int* a = array;
    for(i = 0; i < len; i++) {
        sum += *a * len;
        a++;
    }
    printf(“%s”, string);
}
```

ÜBUNGSZETTEL 1



- RISC-V Assembler
- Wie immer zu finden in Moodle